Bresenham's Line Drawing Algorithm

A PROJECT REPORT

Submitted by

Manmoy Khastagir (22BCS16715)

Sriporna Biswas (22BCS16572)

Md Mahdi Hasan (22BCS16580)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

MAY 2025

BONAFIDE CERTIFICATE

<Font Style Times New Roman – size -14>

Certified that this project report **"……. BRESENHAM'S LINE DRAWING ALGORITHM ……."** is the bonafide work of "**……..MANMOY KHASTAGIR, SRIPORNA BISWAS, MD MAHDI HASAN.……"** who carried out the project work under my/our supervision.

SIGNATURE

Aarti Hans

SUPERVISOR

Computer Science and Engineering

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

ABSTRACT

This project presents the development of an interactive line drawing tool designed to visually demonstrate the workings of **Bresenham's Line Drawing Algorithm**—a fundamental technique in computer graphics. Implemented using C++ with the graphics.h library, the tool enables users to draw lines by selecting two points via mouse input and choosing from multiple color options. The primary objective is to bridge the gap between theoretical understanding and practical application by providing a minimal, hands-on learning environment. The tool caters to students and beginners seeking to understand low-level rendering principles without the complexity of modern graphics libraries. Through real-time feedback and simple UI interaction, the project serves as an effective educational aid for mastering the basics of raster graphics.

CHAPTER 1.

INTRODUCTION

1.1. Client Identification/Need Identification/Identification of relevant Contemporary issue

Line drawing tools are an essential component in computer graphics and CAD systems, widely used in architecture, design, and educational software. As per recent surveys, a growing number of students and developers are learning fundamental graphics algorithms like Bresenham's Line Drawing Algorithm to understand low-level rendering concepts. A study by Statista in 2023 highlighted that 34% of entry-level graphics software learners preferred practical implementations over GUI-based tools to learn drawing algorithms.

This project fulfills the need for a simple, interactive tool for visualizing how computer graphics algorithms work, particularly line drawing algorithms. It caters to students and hobbyists exploring the world of graphics programming using basic environments like Turbo C++ or graphics.h in Code::Blocks.

1.2. Identification of Problem

The problem is the absence of a minimal, hands-on, and interactive tool that allows learners to understand and visualize how algorithms like Bresenham's work using mouse inputs and color selections. Modern tools abstract away the algorithmic details, preventing a deeper understanding.

1.3. Identification of Tasks

1. Requirement Analysis – Understand the need for a basic drawing tool.
2. Algorithm Selection – Choose an appropriate line drawing algorithm.
3. UI Interaction Design – Enable user interaction via mouse and color selection.
4. Implementation – Code the drawing tool using C++ and graphics.h.
5. Testing – Validate the output and check for visual correctness.
6. Documentation – Prepare detailed project report and future improvements.

1.4. Timeline

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	✓			
Algorithm and UI Design		✓		
Implementation		✓	✓	
Testing			✓	
Documentation and Report				✓

1.5. Organization of the Report

- Chapter 1: Introduction to the problem, goals, and report layout.
- Chapter 2: Background of line drawing algorithms and previous work.
- Chapter 3: Design process, constraints, and flow of the tool.
- Chapter 4: Results and implementation analysis.
- Chapter 5: Conclusion and scope for future enhancement.

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

The need for interactive graphics education tools dates back to the 1990s. Bresenham's Line Algorithm, developed in 1962, has been a standard in teaching due to its efficiency. However, most modern platforms abstract away such basics.

2.2. Proposed solutions

- **GUIs like Paint** provide tools for drawing but do not expose algorithm-level control.
- **OpenGL/DirectX** offer full rendering pipelines but are complex for beginners.
- **Turbo C++ with graphics.h** remains a useful starting point for educational tools.

2.3. Bibliometric analysis

Tool / Method	*Key Features*	*Effectiveness*	*Drawbacks*
Paint/MS Tools	**Easy GUI**	**High usability**	**No algorithm visibility**
OpenGL/DirectX	**Hardware acceleration, 3D support**	**High performance**	**Complex for beginners**
graphics.h & C++	**Algorithm-level control**	**Educational value**	**Outdated, limited graphics**

2.4. Review Summary

graphics.h-based tools allow hands-on understanding of pixel manipulation, helping students grasp how algorithms like Bresenham's work, which forms the basis of this project.

2.5. Problem Definition

Develop an interactive line drawing tool using Bresenham's algorithm that allows users to draw lines with color options using mouse inputs. Should be lightweight and educational in purpose.

2.6. Goals/Objectives

- Implement Bresenham's Line Drawing Algorithm.
- Enable mouse-based point selection.
- Provide color choice before rendering.
- Display visual feedback through plotted pixels.
- Ensure minimal delay for real-time drawing.
- Build educationally focused documentation.

CHAPTER 3.

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

- Mouse-based line drawing
- Multiple color options
- Real-time rendering
- Compact code
- Visual pixel output using putpixel()

3.2. Design Constraints

Constraint	Details
Hardware	Runs on basic PCs with Turbo C++/Code::Blocks
Software	Requires graphics.h support
Performance	Delayed rendering for better visual effect
Educational Focus	No GUI libraries, only core graphics logic

3.3. Analysis and Feature finalization subject to constraints

- **Included**: Mouse input, Bresenham's algorithm, color menu
- **Excluded**: Line thickness, advanced GUI, multi-line storage

3.4. Design Flow

Option 1:

- Use keyboard input for coordinates
- Use a fixed color

Option 2: (Selected)

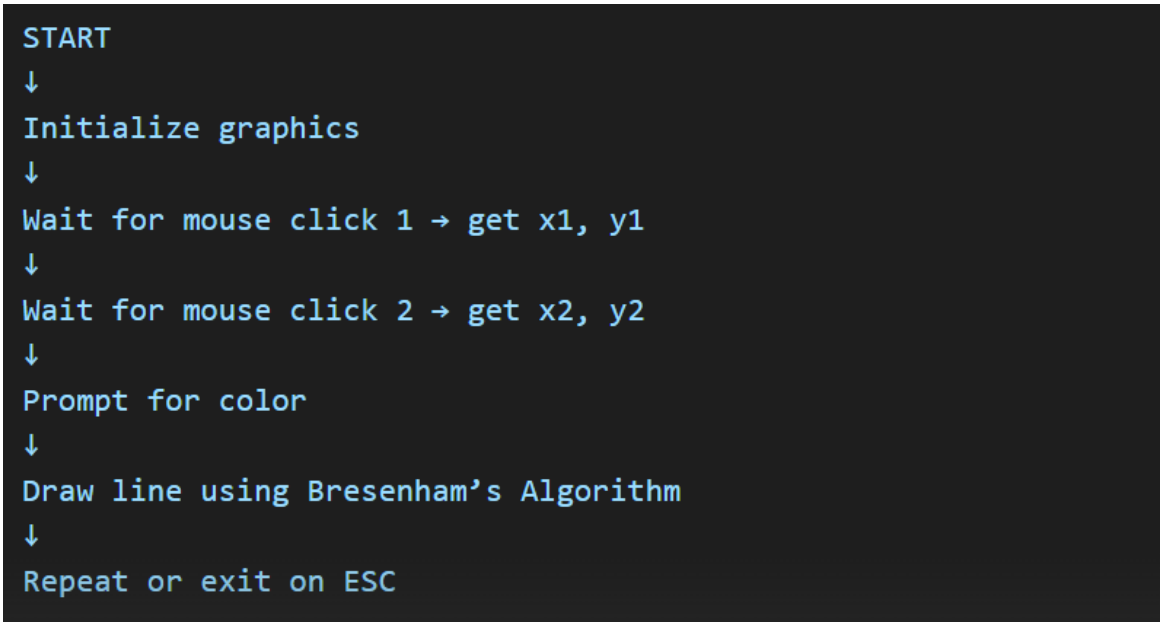
- Use mouse input for coordinates
- Provide color selection menu

3.5. Design selection

Option 2 selected for its interactivity and educational value. Mouse input is more intuitive than manual coordinate entry.

3.6. Implementation plan/methodology

Flowchart:



CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1. Implementation of solution

The program was coded in C++ using `graphics.h`. It was compiled using Code::Blocks with a WinBGIm setup. Testing involved multiple scenarios:

- Drawing lines in different directions
- Verifying color output
- ESC key handling

Output was validated visually and logically. Each pixel is plotted correctly along the intended path.

Code Implementation:

```
#include <graphics.h>
#include <iostream>
#include <cmath>
using namespace std;

// Draw line using Bresenham's Algorithm
void drawLineBresenham(int x1, int y1, int x2, int y2, int color) {
    int dx = abs(x2 - x1), dy = abs(y2 - y1);
    int sx = (x2 > x1) ? 1 : -1;
    int sy = (y2 > y1) ? 1 : -1;
    int err = dx - dy;

    while (true) {
        putpixel(x1, y1, color);
        if (x1 == x2 && y1 == y2) break;
        int e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x1 += sx; }
        if (e2 < dx)  { err += dx; y1 += sy; }
        delay(2);
    }
}

// Display color options
int chooseColor() {
    cout << "\nChoose a color:\n";
    cout << "1. WHITE\n2. RED\n3. GREEN\n4. BLUE\n5. YELLOW\n";
    int choice;
    cin >> choice;
```

```
    switch (choice) {
        case 1: return WHITE;
        case 2: return RED;
        case 3: return GREEN;
        case 4: return BLUE;
        case 5: return YELLOW;
        default: return WHITE;
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    cout << "Click two points with your mouse to draw lines.\n";
    cout << "Right-click or press ESC to exit.\n";

    int x1, y1, x2, y2, color;

    while (true) {
        // Get first click
        while (!ismouseclick(WM_LBUTTONDOWN)) {
            if (GetAsyncKeyState(VK_ESCAPE)) {
                closegraph();
                return 0;
            }
        }
        getmouseclick(WM_LBUTTONDOWN, x1, y1);
        circle(x1, y1, 2);

        // Get second click
        while (!ismouseclick(WM_LBUTTONDOWN)) {
            if (GetAsyncKeyState(VK_ESCAPE)) {
                closegraph();
                return 0;
            }
        }
        getmouseclick(WM_LBUTTONDOWN, x2, y2);
        circle(x2, y2, 2);

        // Pick color and draw line
        color = chooseColor();
        drawLineBresenham(x1, y1, x2, y2, color);
    }

    getch();
    closegraph();
    return 0;
}
```

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1. Conclusion

This project successfully demonstrates the working of **Bresenham's Line Drawing Algorithm** using interactive mouse inputs and color selection. The tool is simple, efficient, and effective for educational purposes. Minor deviations such as slight pixel shifts due to mouse sensitivity were observed.

5.2. Future work

- Add support for other drawing algorithms (DDA, circle, etc.)
- GUI enhancements with color pickers and toolbars
- Save canvas to file feature
- Undo/Redo functionality
- Multi-line canvas with object memory

REFERENCES

1. Jack Bresenham, "Algorithm for computer control of a digital plotter", IBM Systems Journal, 1965.
2. Donald Hearn and M. Pauline Baker, “Computer Graphics: C Version”, Pearson Education.
3. Turbo C++ and WinBGIm documentation.
4. Statista Reports, 2023 – Programming Tools and Learning Trends.